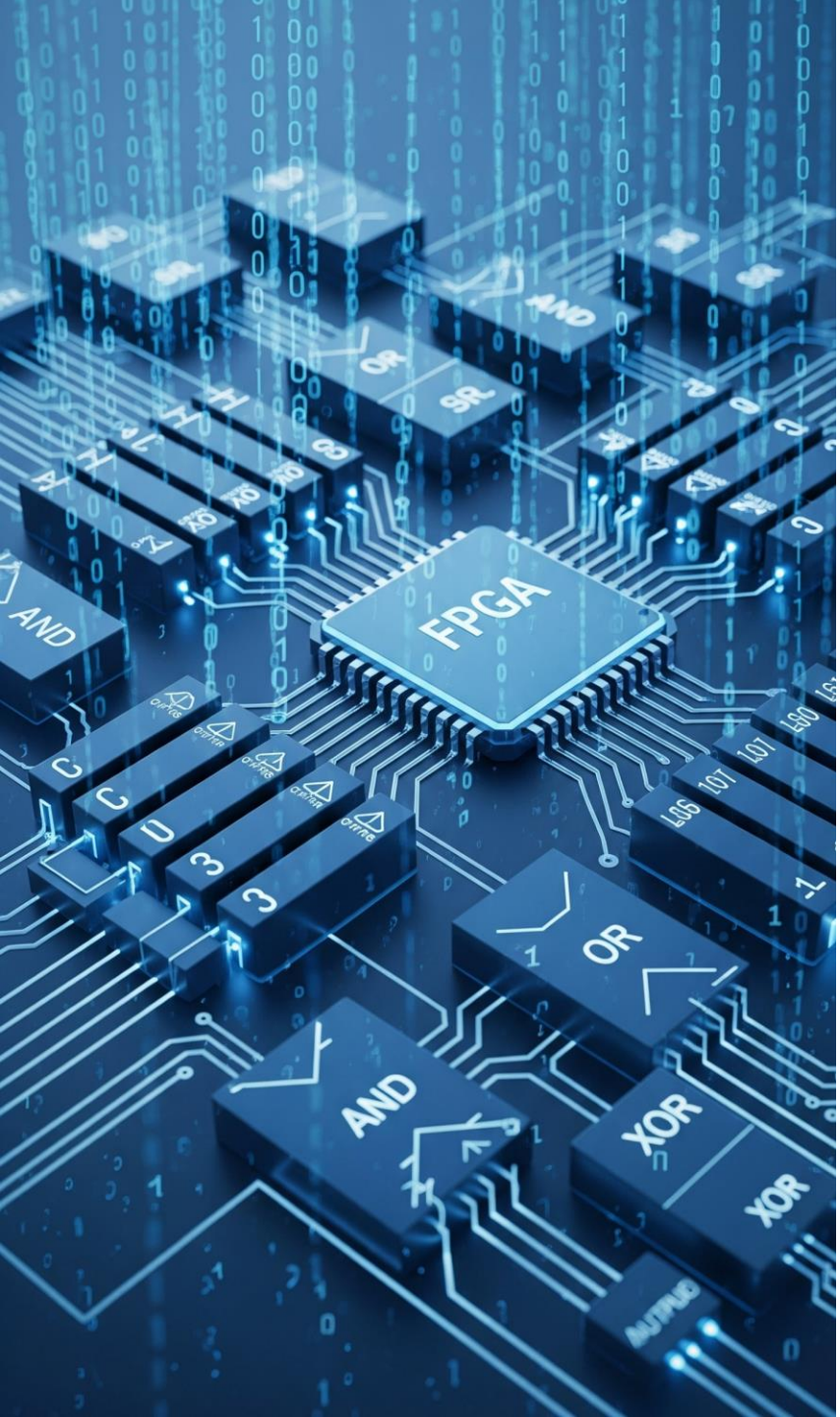


Técnicas Digitales 1

Máquinas de Estado Finito – FSM

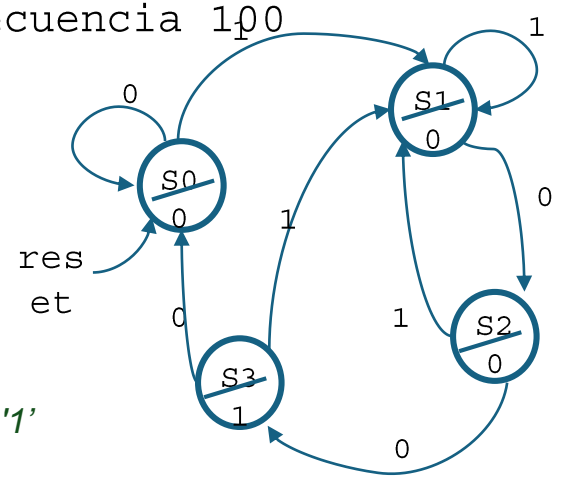
Autómatas de Moore y Mealy

VERILOG

Ing. Jorge Nicolás Mercado

Descripción Verilog FSM Moore

Diagrama de estados para FSM Moore. Secuencia 100



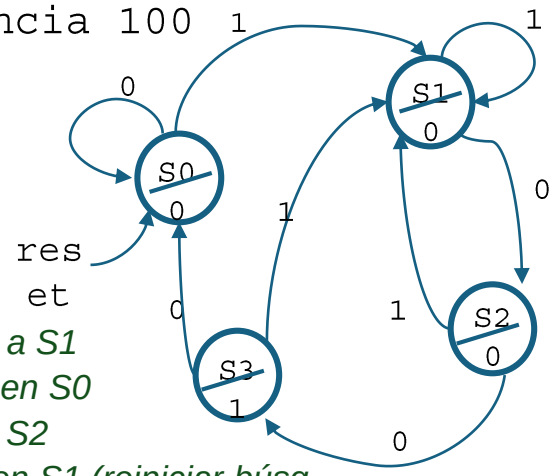

```
module detector_secuencia_moore(
    input clk,
    input reset,
    input X,
    output reg Z
);
// Definición de estados
parameter [1:0] S0 = 2'b00,  // Estado inicial - esperando primer '1'
                S1 = 2'b01,  // Se detectó '1' - esperando '0'
                S2 = 2'b10,  // Se detectó '10' - esperando segundo '0'
                S3 = 2'b11;  // Secuencia completa '100' detectada

reg [1:0] estado_actual;
reg [1:0] estado_siguiente;

 // Lógica de siguiente estado - se ejecuta en cada flanco de reloj
    always @(posedge clk or posedge reset)
        if (reset) estado_actual <= S0;                    // Si hay reset, volver al estado inicia
l S0
        else       estado_actual <= estado_siguiente;    // Transición al siguiente estado
```

Descripción Verilog FSM Moore

Diagrama de estados para FSM Moore. Secuencia 100

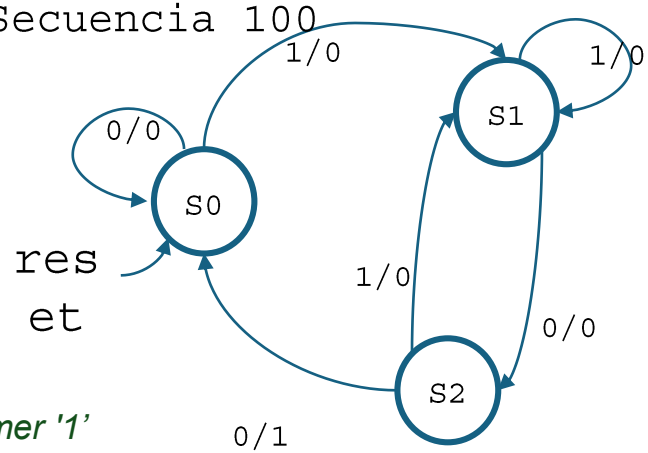



```
// Lógica de transición de estados - combinacional
always @(*)
    case (estado_actual)
        S0: if (X == 1'b1) estado_siguiente = S1;    // Si detectamos '1', ir a S1
                else       estado_siguiente = S0;    // Si es '0', permanecer en S0
        S1: if (X == 1'b0) estado_siguiente = S2;    // Si detectamos '0', ir a S2
                else       estado_siguiente = S1;    // Si es '1', permanecer en S1 (reiniciar búsq
ueda)
        S2: if (X == 1'b0) estado_siguiente = S3;    // Si detectamos '0', secuencia completa, ir
a S3
                else       estado_siguiente = S1;    // Si es '1', volver a S1 (tenemos nuevo '1')
        S3: if (X == 1'b1) estado_siguiente = S1;    // Si nuevo bit es '1', ir a S1 (iniciar nueva s
ecuencia)
                else       estado_siguiente = S0;    // Si es '0', ir a S0 (reiniciar completamente)
         default: estado_siguiente = S0;             // Por defecto, volver al estado inicial
     endcase

// Lógica de salida - solo depende del estado actual
always @(*)
     case (estado_actual)
        S3: Z = 1'b1;            // Solo activar salida cuando estamos en estado S3
         default: Z = 1'b0;      // En todos los demás estados, salida en '0'
     endcase
endmodule
```

Descripción Verilog FSM Mealy

Diagrama de estados para FSM Mealy. Secuencia 100



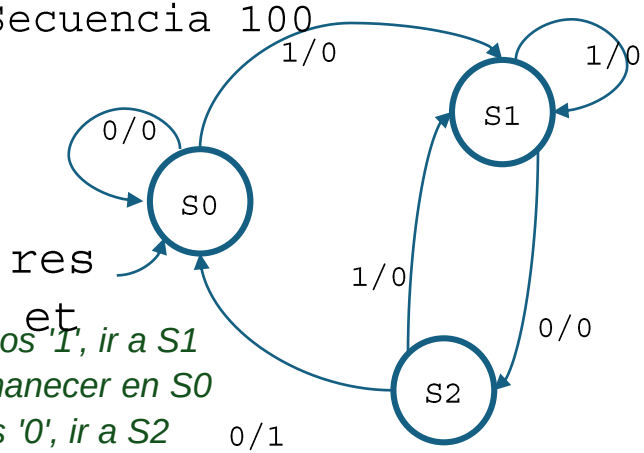

```
module detector_secuencia_mealy(
    input clk,
    input reset,
    input X,
    output reg Z
);
// Definición de estados
parameter [1:0] S0 = 2'b00,  // Estado inicial - esperando primer '1'
                S1 = 2'b01,  // Se detectó '1' - esperando '0'
                S2 = 2'b10,  // Se detectó '10' - esperando segundo '0'

reg [1:0] estado_actual;
reg [1:0] estado_siguiente;

 // Lógica de siguiente estado - se ejecuta en cada flanco de reloj
    always @(posedge clk or posedge reset)
        if (reset) estado_actual <= S0;                    // Si hay reset, volver al estado inicia
l S0
        else       estado_actual <= estado_siguiente;    // Transición al siguiente estado
```

Descripción Verilog FSM Mealy

Diagrama de estados para FSM Mealy. Secuencia 100




```
// Lógica de transición de estados - combinacional
always @(*)
    case (estado_actual)
      S0: if (X == 1'b1) estado_siguiente = S1;    // Si detectamos '1', ir a S1
            else         estado_siguiente = S0;    // Si es '0', permanecer en S0
      S1: if (X == 1'b0) estado_siguiente = S2;    // Si detectamos '0', ir a S2
            else         estado_siguiente = S1;    // Si es '1', permanecer en S1 (reiniciar búsq
ueda)
      S2: if (X == 1'b0) estado_siguiente = S0;    // Si detectamos '0', secuencia completa, ir
a S0
            else         estado_siguiente = S1;    // Si es '1', volver a S1 (tenemos nuevo '1')
      default: estado_siguiente = S0;              // Por defecto, volver al estado inicial
    endcase

// Lógica de salida - depende del estado actual y la entrada
always @(*)
          Z = 1'b0; // Inicializar salida en '0'
    // Detectar cuando estamos en estado S2 Y la entrada es '0'
    // Esto equivale a: estado=S2 (10) y entrada=0 → secuencia "100" completa

     if (estado_actual == S2 && X == 1'b0)  Z = 1'b1;      // activa salida cuando se completa y
"X"=0
         default: Z = 1'b0;                                // En todos los demás estados, salid
a en '0'
```